

Machine Learning Technologies

Classification

Logistic Regression

Donny Hurley

ATU Galway

Now some different problems:

- With regression, we assumed we were taking input and producing a quantity: what number (continuous value) do we think y will be (close to) for a particular input?
- Many ML problems have a more limited set of outputs. Often the question is more like: which group is this input (mostly likely) part of?

These are *classification* problems.

Our outputs will be restricted to the possible classes for our problem. i.e. for n samples with K categories, we predict a vector of n values chosen from $C_1, C_2, \dots, C_k$. Our inputs will stay the same: an $n \times m$ matrix of numbers (for n samples and m features).

Some problems that can be seen as classification:

- Is this email spam? Is this credit card charge fraud? (2 categories)
- What (English) word did the user say into the microphone? ($\approx 470\text{k}$ categories)
- What day will ATU Galway open next year? ($\ll 365$ categories) Or is that regression?
- How many people are in this picture? Who are the people?
- Given an RGB colour value, what colour word describes it?
- Given weather observations, can you guess what city are we looking at?

- There are many ways to model classification.
- All do the same basic job:
 - 1 train on known data
 - 2 then make predictions about correct category for other inputs.

```
1 model = SomeClassifierModel (....)
2 model.fit(X_train , y_train)
3 y_predicted = model.predict(X_test)
4 somehow_compare_results(y_test , y_predicted)
```

(Often use validation to pick the best form of the Classifier, previously used

```
1 model = LinearRegression (....)
```

it all takes the same basic idea in code)

Evaluating Classification Performance

For evaluating Classification algorithms often people just see how accurate the predictions are on the test set

```
1 y_pred = model.predict(X_test)
2 from sklearn.metrics import accuracy_score
3 accuracy_score(y_test, y_pred)
```

Will give a score (< 1) which is the fraction of how many predictions were correct.

This stores the y predictions separately (which may be useful later) but the model has a method itself that can be used if you just want the accuracy score

```
1 model.score(X_test, y_test)
```

This one line does the predicting and comparing of predictions in the same step. We will come back to this later, about alternatives.

Definition

A **Binary** *classification problem* has:

- *Negative Class*
- *Positive Class*

$$y \in [0, 1] \text{ or } y \in [-1, 1]$$

We'll consider Multi-class problems later. Some models can directly address multi-class (Naïve Bayes, k -nearest neighbours) while some are inherently binary but can be "extended" (SVM, Logistic Regression)

Methods for use when the response variable ($\hat{y}$) has only two possible outcomes: a customer buys or does not buy a product, a patient lives or dies, or a candidate accepts or declines a job offer.

In general, we call the two outcomes of the response variable "success" and "failure" and represent them by 1 and 0. The mean is then the proportion of 1s.

To start on classification, do we really need a different algorithm? Let's see if we can "tweak" what we know about regression.

Note:

- Logistic Regression is a confusing name because we are not trying to predict a continuous value.
- We do predict a continuous value but we then threshold it to take on discrete (non-continuous) values.
- Logistic Regression, in the end, is actually a classification algorithm.

Despite its name, Logistic Regression is a classification ML model. Something is either in category 0 or 1 and the *predicted* value is between 0 and 1. The predicted value is a *probability*

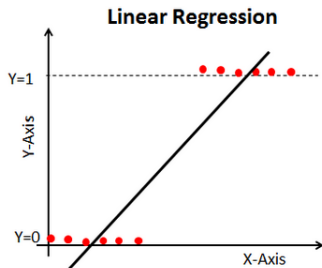
The response variable y is either 0 or 1. By our standard formulae for regression you will recall it is

$$\hat{y} = w_0 + w_1 x_1$$

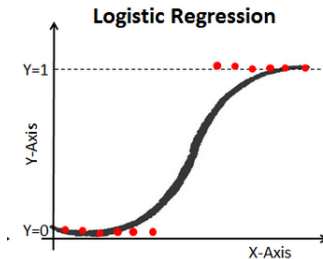
The estimation of $\hat{y}$ has a good chance of being < 0 or > 1 . We want an estimation for p between 0 and 1.

Logistic Regression

Additionally we get a plot (only one feature) like



The picture clearly is not going to be very accurate. We would much rather a picture like



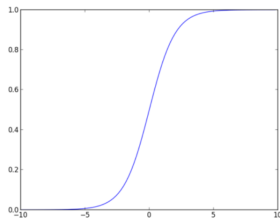
Model for Logistic Regression

Sigmoid function, also called the logistic function, takes any real-valued number and maps it to a value between 0 and 1. If the curve tends towards $+\infty$, y goes towards 1 and if it tends towards $-\infty$, y goes towards 0

$$f(x) = \frac{1}{1 + e^{-x}}$$

Note: Sigmoid function has regularly been used in the past for Neural Networks, Deep Learning, the main one in the past

We have the function that gives us the shape we want, how do we relate this to our $\hat{y} = w_0 + w_1x$?



Definition

Logit Function

$$\text{logit}(p) = \log \left(\frac{p}{1-p} \right)$$

where $\log$ is the **natural logarithm** (probably **ln** on your calculator, $\log$ with base e (exponential)).

Inverse logit is the sigmoid function and

$$\frac{\exp(x)}{1 + \exp(x)} = \frac{1}{1 + \exp(-x)}$$

We use the sigmoid function

$$\frac{1}{1 + \exp(-x)}$$

to bind our values between 0 and 1, then (log means natural log)

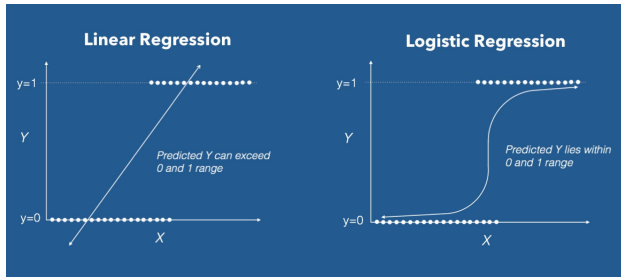
$$\log\left(\frac{y}{1-y}\right) = w_0 + w_1x_1 + \dots + w_nx_n.$$

where $\mathbf{x}$ describes the features of a datapoint. So

$$f_{\mathbf{w}}(\mathbf{x}) = \hat{y} = \frac{1}{1 + e^{-(w_0 + w_1x_1 + \dots + w_nx_n)}}$$

Logistic Regression

Despite its name, Logistic Regression is a classification ML model.



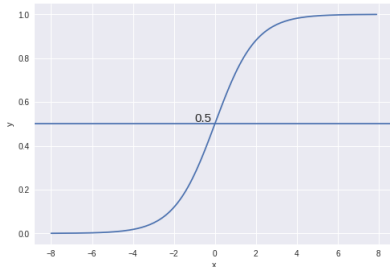
Something is either in category 0 or 1 and the *predicted* value is between 0 and 1. The predicted value is a *probability*

Decision Boundary

We expect our classifier to give us a set of outputs or classes based on probability when we pass the inputs through a prediction function and returns a probability score between 0 and 1.

Example

We have 2 classes, cats and dogs (1 - dog , 0 - cats). We basically decide with a threshold value above which we classify values into Class 1 and of the value goes below the threshold then we classify it in Class 2.



Chosen the threshold as 0.5: if the prediction function returned a value of 0.7 then we would classify this observation as Class 1(DOG). If our prediction returned a value of 0.2 then we would classify the observation as Class 2(CAT).

We can choose a different boundary to adjust for FP vs FN.

- The end result of our ML system (for a binary classifier) is either the number 0 or 1.
- ML systems will often only output the class it thinks is most likely with this threshold, i.e. if one side of decision boundary always output 0, if the other output 1
- A lot of ML libraries that deal with Logistic Regression you can retrieve the “probability” for its result, however, some may just output 0 or 1.

- Clearly this is a binary classification system, so what happens if we have multiple categories?
- For now we'll just take the scikit learn method as “magic” in how it deals with multiple categories. I'll explain how this works later.
- A version of Logistic Regression is very often applied on the last layer of a neural network so understanding it will help when we get to that later.

Loss Function

To train a logistic regression model, we have to figure out what weights ($\mathbf{w}$) minimise the cost/loss. We have to choose the cost function

Mean Squared Error performs very poorly for Logistic Regression

$$\frac{1}{2m} \sum_{i=1}^m \left(y^{(i)} - f_{\mathbf{w}}(x^{(i)}) \right)^2$$

it would end up being a non-convex function with many local minimum (so difficult to find the global minimum) instead use this Cost function

$$\mathcal{L}(f_{\mathbf{w}}(\mathbf{x}), y) = \begin{cases} -\log(f_{\mathbf{w}}(\mathbf{x})) & \text{if } y = 1 \\ -\log(1 - f_{\mathbf{w}}(\mathbf{x})) & \text{if } y = 0 \end{cases}$$

the loss for one datapoint.

Loss Function (Log-Loss)

This is not differentiable as stated so let's combine it to

$$\mathcal{L}(f_{\mathbf{w}}(\mathbf{x}), y) = -y \log(f_{\mathbf{w}}(\mathbf{x})) - (1 - y) \log(1 - f_{\mathbf{w}}(\mathbf{x}))$$

Of course we need to do this over all samples. Therefore we use the following as our Loss Function for logistic regression

$$L(\mathbf{w}) = -\frac{1}{m} \left[\sum_{i=1}^m -y^{(i)} \log(f_{\mathbf{w}}(\mathbf{x}^{(i)})) - (1 - y^{(i)}) \log(1 - f_{\mathbf{w}}(\mathbf{x}^{(i)})) \right]$$

The function is convex which greatly simplifies minimisation. We need to find the optimum parameters $\mathbf{w}$ that minimises the loss on the training set. Then we can use the model to predict new outputs on unseen data.

The algorithm to solve the problem and find the $\mathbf{w}$ is the same as others and we'll, again, see it later.

```
1 from sklearn.linear_model import LogisticRegression
```

It has some options, but we'll mostly just keep the default

```
1 penalty : {'l1', 'l2', 'elasticnet', None}, default='l2'
```

is a hyperparameter for it, the default is good - I'm just pointing it out, I'll explain later.

```
1 max_iter: int, default=100
```

I've mentioned a few times about how the learning is doing things over and over again. Here we can control the max number of times it will do this process. Sometimes you might need more.

Now to talk about the outputs. Like LinearRegression it has

```
1 model.coef_  
2 model.intercept_  
3 model.score(X_test, y_test) # Gives the "accuracy" score of the model  
4 y_pred = model.predict(X_test)
```

A new one it has is

```
1 model.classes_
```

This gives you a list of class labels known to the model. e.g. your y's are

```
1 y = np.array([0,1,2,1,1,1,1,1,0,2,1]) #or a pandas series equivalent
```

then `model.classes_` will be `[0,1,2]`. If your y's are

```
1 y = np.array(["cat","dog","pikachu","dog","dog","dog","cat","pikachu"])
```

then `model.classes_` will be `["cat","dog","pikachu"]` instead.

A new method it has is

```
1 model.predict_proba(X_test)
```

This one gives probability estimates for all the classes. Maybe you want to know how confident the model is in its predictions - but be aware, models can be over confident!

The `.predict` I mentioned on the previous slide uses thresholds to make the decisions.

You may also want the probabilities for some performance metrics when trying to make decisions about models.

Transform Data

Sometimes the data we have is not in the best form for the model we are training, so we need to change it. However

- We don't alter the entire data set
- This would mean any future data being evaluated by the model would also first have to be transformed
- We build the transform into the model using **make_pipeline**

```
1 from sklearn.pipeline import make_pipeline
2
3 model = make_pipeline(
4     MinMaxScaler(),
5     LogisticRegression()
6 )
7 model.fit(X_train, y_train)
8 model.score(X_test, y_test)
```

Any data that goes into this pipeline has the first step: the data is transformed, then the transformed data goes into the LogisticRegression step. We build the transformation into the model. We can chain more things together in a pipeline.

The example I had on the previous slide was `MinMaxScaler` which is a very commonly used transformation step, <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html>.

It makes all the data in the same range, by default the minimum value is changed to 0, the maximum to 1 and everything else scaled accordingly.